

Name: Rhilo Sotto  
RED ID: 130551574  
Date: 10/25/2024

# Lab 5 Report

## 1. What does your program do?

The program generates musical notes depending on the keypad button pressed via PWM waves within an infinite loop. The program runs continuously, and since there is no break statement, it will run indefinitely until power is lost.

## 2. What variables/registers does your program use?

The program uses the **DDRC** register to set PINC5 as an output for the PWM.

The **DDRD** register is used to set the rows of the GPIO keypad to outputs

The **DDRB** register is used to set the columns of the GPIO keypad to inputs

The **PORTC** register is used to set the what value is being outputted to PINC5, either high or low.

The **PORTD** register is used to set rows to logic high and isolate the row of a button pressed during iterative scanning.

The **PORTB** register is used to enable the pull-up resistor property and make the default input to the port logic high, and logic low when the keypad button is pressed.

The **PINB** register is used to read the Port B input pins during iterative scanning.

The **TCCR0A** register to set Timer 0 to CTC mode.

The **TCCR0B** register to set the prescaler to x256 and start Timer 0, where with a clock frequency of 16MHz corresponds to a period of 4096 microseconds, or 4.096 milliseconds.

The **OCR0A** register acts as Timer 0's max value with which the counter would reset to 0 upon reaching, effectively the fraction of the pre-scaled period that Timer 0 would elapse before resetting.

The **OCR0B** register acts as Timer 0's duty cycle, having a value that is half of OCR0A that is how long the program should output High for.

The **TIFR0** register reads OCR0A and OCR0B's overflow flags and allows the program to wait for a period of time before switching from output high to low.

The **keynote** 2D array is used to efficiently map the row and column of the GPIO keypad to the musical note associated with it in the form of the note's period to be stored in OCR0A, the values are pre-calculated using the period formula provided in the lab description multiplied by the clock frequency, divided by the prescale value, subtracting by one to account for counting from 0, and rounding to the nearest integer.

$$OCRA(x) = \text{round}\left(\frac{1}{440 \cdot 2^{\frac{x}{12}}} * \frac{f_{cpu}}{PS} - 1\right)$$

### 3. What is the main algorithm/logic of your program?

The main program calls the **init** function to initialize the DDRC register to set PIN5 as an output.

Then it calls the **GPIO\_init** function to initialize the Ports D and B to the settings used in Lab 3, pull-up input/output respectively.

Then it calls the **timer\_init** function to set Timer 0's mode to CTC which will cause the clock to reset upon reaching the value stored in OCR0A. TCCR0B is used to set the prescaler value to x256 and start the timer, which was chosen since the longest period note required is the 440Hz middle A note, which is approximately 2.273ms and can be achieved the max period of 4.096ms in CTC mode with the most accuracy.

The **generate\_PWM** function makes PIN5 output high until OCR0B's overflow flag is raised, resets the flag and sets the output to low until OCR0A's overflow flag is raised. OCR0A and OCR0B are set before the function is called in main, and OCR0B is set to about half of OCR0A.

The program enters the infinite while loop, where it will iteratively scan the GPIO keyboard using the same algorithm as Lab 3. If a GPIO keypad button is detected to be pressed, OCR0A is set to the corresponding value in the **keynote** array, which contains the period of the note to be played (calculated beforehand using the above formula for  $f_{cpu} = 16\text{MHz}$ ,  $PS = 256$ , and  $x$  as the index of the GPIO keypad) and OCR0B is set to half of OCR0A by bit shifting OCR0A one bit to the right. I chose to calculate these values beforehand to make the microcontroller slightly more efficient, and not have to rely on the math.h library and pow functions, which may drastically impact runtime. It then continuously runs **generate\_PWM** until the key that is being pressed is released.

### 4. What external hardware connections did you make?

The GPIO keypad is connected to my microcontroller by pins PD4-PD7, and PB0-PB3, where the rows correspond to port D and columns correspond to port B. I have a wire connecting from pin PC5 to a breadboard that connects the red and white cables of my AUX cable in parallel via another wire so that they share the same output. I have another wire connecting the yellow ground wire of my AUX cable to the GND pin on my microcontroller.

## Appendix (Code)

```
/*
 * Lab5.c
 *
 * Created: 10/4/2024 9:53:03 PM
 * Author : rnsot
 */

#define F_CPU 16000000UL // 16MHz clock speed

#include <avr/io.h>

void init(void) {
    DDRC |= (1 << DDRC5); // output pin PC5
}

void GPIO_init(void) {
    // Rows = PORTD, Columns = PORTB
    // set rows to output
    DDRD |= (1<<DDD4 | 1<<DDD5 | 1<<DDD6 | 1<<DDD7); // Port D (4-7)
    // set column to input
    DDRB &= ~(1<<DDB0 | 1<<DDB1 | 1<<DDB2 | 1<<DDB3); // Port B (0-3)
    // pull up property enabled (columns)
    PORTB |= (1<<PORTB0 | 1<<PORTB1 | 1<<PORTB2 | 1<<PORTB3);
    // rows to logic high
    PORTD |= (1<<PORTD4 | 1<<PORTD5 | 1<<PORTD6 | 1<<PORTD7);
}

void timer_init(void) { // initialize timer settings
    // F_CPU = 16MHz, want (max) 2.273ms
    // ps = 1, NM -> 16us = 0.016ms
    // ps = 8, NM -> 128us = 0.128ms
    // ps = 64, NM -> 1024us = 1.024ms
    // ps = 256, NM -> 4096us = 4.096ms **
    // ps = 1024, NM -> 16384us = 16.384ms

    /*
    want PERIOD given PS = x256:
    calculate wanted period per note (1/(440*2^(x/12)))
    x | us*10    us        ms  steps  -1
A4   0 | 22727 = 2273 = 2.273 = 142 = 141
A#4  1 | 21452 = 2145 = 2.145 = 134 = 133
    */
}
```

|     |                                      |
|-----|--------------------------------------|
| B4  | 2   20248 = 2025 = 2.025 = 127 = 126 |
| C5  | 3   19111 = 1911 = 1.911 = 119 = 118 |
| C#5 | 4   18039 = 1804 = 1.804 = 113 = 112 |
| D5  | 5   17026 = 1703 = 1.703 = 106 = 105 |
| D#5 | 6   16071 = 1607 = 1.607 = 100 = 99  |
| E5  | 7   15169 = 1517 = 1.517 = 95 = 94   |
| F5  | 8   14317 = 1432 = 1.432 = 89 = 88   |
| F#5 | 9   13514 = 1351 = 1.351 = 84 = 83   |
| G5  | 10   12755 = 1276 = 1.276 = 80 = 79  |
| G#5 | 11   12039 = 1204 = 1.204 = 75 = 74  |
| A5  | 12   11364 = 1136 = 1.136 = 71 = 70  |
| A#5 | 13   10726 = 1073 = 1.073 = 67 = 66  |
| B5  | 14   10124 = 1012 = 1.012 = 63 = 62  |
| C6  | 15   09556 = 0956 = 0.956 = 60 = 59  |

\*/

// Set to CTC Mode

TCCR0A |= (1 << WGM01);

// Pre-scaler set to 256, STARTS the timer

TCCR0B |= (1 << CS02);

}

void generate\_PWM(void) { // generates a PWM waveform with variable duty cycle

PORTC |= (1 << PORTC5); // HIGH

while ( (TIFR0 & (1 << OCF0B)) == 0 ) {} // busy while until OCF0B flag == 1 in TIFR0

register

TIFR0 |= (1 << OCF0B); // reset OCR0B overflow

PORTC &= ~(1 << PORTC5); // LOW

while ( (TIFR0 & (1 << OCF0A)) == 0 ) {} // busy while until OCF0A flag == 1 in TIFR0

register

TIFR0 |= (1 << OCF0A); // reset OCR0A overflow flag

}

int main(void) {

init();

GPIO\_init();

timer\_init();

unsigned int keynote[4][4] =

{ // pre-calculated periods

```

        {141, 133, 126, 118},
        {112, 105, 99, 94},
        {88, 83, 79, 74},
        {70, 66, 62, 59}
    };

    while (1) {
        /* iterative scan of GPIO keypad */
        for (char i = 4; i < 8; ++i) {
            // set row[i] low
            PORTD &= ~(1<<i);
            for (char j = 0; j < 4; ++j) {
                // check column[j] if low
                if ( !(PINB & (1<<j)) ) { // first transition low (on)
                    OCR0A = keynote[i-4][j]; // set period (frequency) based on
index
                    OCR0B = OCR0A >> 1; // 50% duty cycle
                    while ( !(PINB & (1<<j)) ) { // wait until button is released
                        generate_PWM(); // output at PINC5
                    }
                }
            }
            // set row[i] high
            PORTD |= (1<<i);
        }
    }
}

```